On-device intrusion detection for smart-home IoT

Smart-home devices such as cameras, plugs and speakers are cheap, rarely patched, and easily recruited into botnets like Mirai. Detecting these attacks in the cloud or on a central gateway adds latency, depends on connectivity, and exposes household traffic. Running the detection on the device itself is attractive, but it has to fit into very small memory, compute and energy budgets.

Most public datasets make this harder than it first looks. They are network-based, collected from a device in the middle (a hub, a gateway, or a tap) that receives all the traffic. They are really meant for gateway detection, where the goal is to spot which device's traffic looks wrong. On-device detection is a different and more complicated problem, because the device does not see the full traffic the way a gateway does. By default it mostly works with its own application data, and the protocol or network-layer details that an attack usually shows up in are not exposed to the application. For example, the device sends or receives a temperature reading of 23.4, not the packet, flag or protocol information around it. On the device we can measure things like how many messages per minute, how many failed connections, and how often it reconnected, but that may not be enough to detect an attack. Depending on the network stack and libraries, the firmware can be instrumented to see packet-level details too, though collecting them takes extra work. Meanwhile, the public datasets contain the richer network-level information (flags, connection counts, packet sizes), precisely because they were recorded by a device in the middle that saw everyone's envelopes.

So, to detect attacks on the device, we first have to decide which features the device can actually see, and prepare those features before deploying the model. There are two steps here that are not the same: deriving device-level features offline from a public packet capture, and actually measuring those features live on the MCU's own firmware.

Scope. We focus on IP-based Wi-Fi devices, using only the traffic one device sends and receives. BLE and Zigbee devices are excluded because most of the public datasets are IP-based (Wifi, ethernet) and supporting several protocol families would widen the scope too much. A single device can see attacks aimed at itself, such as floods and scans, but not attacks between other devices. The real question, then, is this: with limited visibility, limited control over the network, and limited memory, how can a device run detection on itself, and which models work best under those limits?

Gap. IoT intrusion detection is often evaluated on traffic captured at a gateway, or on pre-extracted feature datasets. But the features a small IoT device can collect from its own traffic differ a lot from those a gateway sees. It is not well understood how detection models perform when restricted to device-local features, what they cost on real embedded hardware, and how this

compares with gateway-side detection. This thesis investigates these questions through a controlled comparison of device-level and gateway-level detection.

Research question. Which lightweight detection models stay accurate and affordable on a real embedded device using only its own traffic, and how does this compare with gateway-side detection?

Rough plan.

- Datasets: train and evaluate on CICIoT2023, using features limited to what a device can compute from its own packets. If time allows, test on a second dataset (IoT-23, Edge-IIoTset or Kitsune) to check how well it carries over to a different network.
- Models: start with a statistical baseline (z-score) and a small decision tree, and possibly others (isolation forest, a small autoencoder). A simple rate-limit threshold is kept as a separate non-ML baseline.
- Hardware: an ESP32-class device, and a Raspberry Pi as the gateway for comparison. Measure accuracy, false alarms, detection delay, RAM, flash, latency and energy.

Public datasets found. All are IP-based (Wi-Fi or Ethernet); none cover BLE, Zigbee or Z-Wave. The ones with raw packet captures are the most useful here, because the features a single device would see can be rebuilt from them.

- CICIoT2023 — network traffic, 33 attacks, has pcaps. Main training set.
- N-BaIoT — per-device network statistics, 10 attacks (Mirai, BASHLITE), made for anomaly detection, no pcaps.
- IoT-23 — network traffic, 20 malware captures, has pcaps; infected device is a Raspberry Pi.
- Edge-IIoTset — network traffic plus some sensor data, 14 attacks, has pcaps and MQTT; industrial.
- Kitsune — network traffic, 9 attacks, has pcaps; mostly IP cameras.
- Bezerra (host-based) — network plus device data (CPU, memory), botnet attacks; the only one recorded on the device itself, but very small.
- TON_IoT — network, logs and sensor data, several attacks; industrial and mixed, harder to align.

References.

- Neto et al., CICIoT2023, Sensors 23(13), 2023
- Meidan et al., N-BaIoT, IEEE Pervasive Computing 17(3), 2018
- Mirsky et al., Kitsune, NDSS 2018
- Anthi et al., IEEE IoT Journal 6(5), 2019
- Bezerra et al., host-based IoT dataset, SBSeg 2018

- Garcia et al., IoT-23, Zenodo, 2020
- Ferrag et al., Edge-IIoTset, IEEE Access 10, 2022
- Ortiz-Garcés et al., Internet of Things 39, 2026
- Hakim et al., arXiv:2607.00553, 2026 (preprint)
- Albaiz & Amsaad, arXiv:2604.08581, 2026 (preprint)
- Trilles et al., Internet of Things 25, 2024